JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class DoubleSummaryStatistics

java.lang.Object
    java.util.DoubleSummaryStatistics

All Implemented Interfaces:

DoubleConsumer

```
public class DoubleSummaryStatistics
extends Object
implements DoubleConsumer
```

A state object for collecting statistics such as count, min, max, sum, and average.

This class is designed to work with (though does not require) streams. For example, you can compute summary statistics on a stream of doubles with:

```
DoubleSummaryStatistics stats = doubleStream.collect(DoubleSummaryStatistics::new,
                                                     DoubleSummaryStatistics::acce
                                                     DoubleSummaryStatistics::comb
```

DoubleSummaryStatistics can be used as a reduction target for a stream. For example:

```
DoubleSummaryStatistics stats = people.stream()
    .collect(Collectors.summarizingDouble(Person::getWeight));
```

This computes, in a single pass, the count of people, as well as the minimum, maximum, sum, and average of their weights.

Implementation Note:

This implementation is not thread safe. However, it is safe to use `Collectors.toDoubleStatistics()` on a parallel stream, because the parallel implementation of `Stream.collect()` provides the necessary partitioning, isolation, and merging of results for safe and efficient parallel execution.

Since:

1.8

Constructor Summary

Constructors

Constructor	Description
`DoubleSummaryStatistics()`	Construct an empty instance with zero count, zero sum, `Double.POSITIVE_INFINITY` min, `Double.NEGATIVE_INFINITY` max and zero average.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`void`	`accept(double value)`	Records another value into the summary information.
`void`	`combine(DoubleSummaryStatistics other)`	Combines the state of another `DoubleSummaryStatistics` into this one.
`double`	`getAverage()`	Returns the arithmetic mean of values recorded, or zero if no values have been recorded.
`long`	`getCount()`	Return the count of values recorded.
`double`	`getMax()`	Returns the maximum recorded value, `Double.NaN` if any recorded value was NaN or

		Double.NEGATIVE_INFINITY if no values were recorded.
double	getMin()	Returns the minimum recorded value, Double.NaN if any recorded value was NaN or Double.POSITIVE_INFINITY if no values were recorded.
double	getSum()	Returns the sum of values recorded, or zero if no values have been recorded.
String	toString()	Returns a string representation of the object.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.function.DoubleConsumer

andThen

Constructor Detail

DoubleSummaryStatistics

```
public DoubleSummaryStatistics()
```

Construct an empty instance with zero count, zero sum, Double.POSITIVE_INFINITY min, Double.NEGATIVE_INFINITY max and zero average.

Method Detail

accept

```
public void accept(double value)
```

Records another value into the summary information.

Specified by:

`accept` in interface `DoubleConsumer`

Parameters:

`value` - the input value

combine

```
public void combine(DoubleSummaryStatistics other)
```

Combines the state of another `DoubleSummaryStatistics` into this one.

Parameters:

`other` - another DoubleSummaryStatistics

Throws:

`NullPointerException` - if other is null

getCount

```
public final long getCount()
```

Return the count of values recorded.

Returns:

the count of values

getSum

```
public final double getSum()
```

Returns the sum of values recorded, or zero if no values have been recorded. If any recorded value is a NaN or the sum is at any point a NaN then the sum will be NaN.

The value of a floating-point sum is a function both of the input values as well as the order of addition operations. The order of addition operations of this method is intentionally not defined to allow for implementation flexibility to improve the speed and accuracy of the computed result. In particular, this method may be implemented using compensated summation or other technique to reduce the error bound in the numerical sum compared to a simple summation of `double` values.

API Note:

Values sorted by increasing absolute magnitude tend to yield more accurate results.

Returns:

the sum of values, or zero if none

getMin

```
public final double getMin()
```

Returns the minimum recorded value, `Double.NaN` if any recorded value was NaN or `Double.POSITIVE_INFINITY` if no values were recorded. Unlike the numerical comparison operators, this method considers negative zero to be strictly smaller than positive zero.

Returns:

the minimum recorded value, Double.NaN if any recorded value was NaN or Double.POSITIVE_INFINITY if no values were recorded

getMax

```
public final double getMax()
```

Returns the maximum recorded value, `Double.NaN` if any recorded value was NaN or `Double.NEGATIVE_INFINITY` if no values were recorded. Unlike the numerical comparison operators, this method considers negative zero to be strictly smaller than positive zero.

Returns:

the maximum recorded value, Double.NaN if any recorded value was NaN or Double.NEGATIVE_INFINITY if no values were recorded

getAverage

```
public final double getAverage()
```

Returns the arithmetic mean of values recorded, or zero if no values have been recorded. If any recorded value is a NaN or the sum is at any point a NaN then the average will be code NaN.

The average returned can vary depending upon the order in which values are recorded. This method may be implemented using compensated summation or other technique to reduce the error bound in the `numerical sum` used to compute the average.

API Note:

Values sorted by increasing absolute magnitude tend to yield more accurate results.

Returns:

the arithmetic mean of values, or zero if none

toString

```
public String toString()
```

Returns a string representation of the object. In general, the `toString` method returns a string that "textually represents" this object. The result should be a concise but informative representation that is easy for a person to read. It is recommended that all subclasses override this method.

The `toString` method for class `Object` returns a string consisting of the name of the class of which the object is an instance, the at-sign character `@', and the unsigned hexadecimal representation of the hash code of the object. In other words, this method returns a string equal to the value of:

```
getClass().getName() + '@' + Integer.toHexString(hashCode())
```

Returns a non-empty string representation of this object suitable for debugging. The exact presentation format is unspecified and may vary between implementations and versions.

Overrides:

`toString` in class `Object`

Returns:

a string representation of the object.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.